

Agenda.

→ Polymorphism

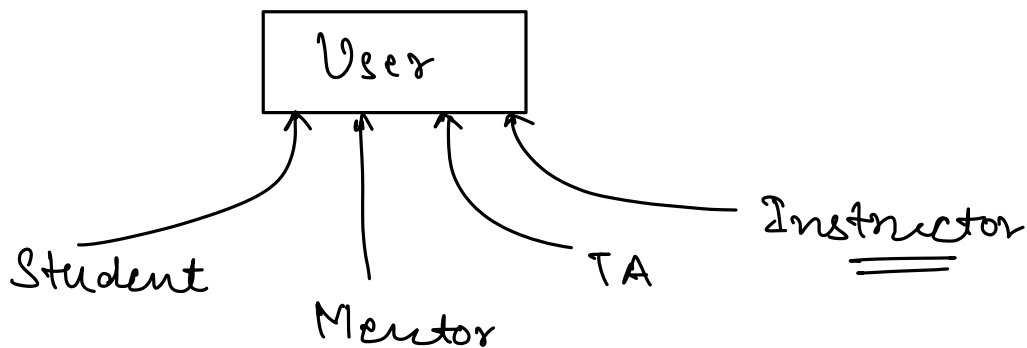
→ Interfaces & Abstract Classes.

→ Static keyword.

Polymorphism

many → forms.

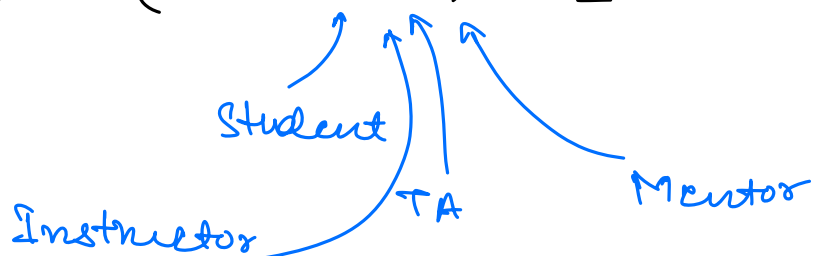
⇒ Something which has many forms.



Student	IS-A	User
Mentor	IS-A	User
TA	IS-A	User
Instructor	IS-A	User

} User has many forms.

changePassword (User user) { ≡ }



```
User user = new Student() ✓  
           = new Mentor() ✓  
           = new Instructor() ✓  
           = new TA() ✓
```

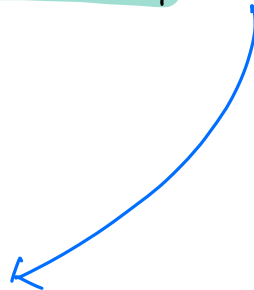
⇒ We can store the object of any child class in the parent class reference, but the vice-versa is NOT possible.

```
Student st = new User() ✗
```

```
PrintNames (List<User> users) {
```

```
    |||  
    |||  
    |||
```

```
}
```



```
users.add(new Student());
```

```
users.add(new Mentor());
```

```
_____  
_____
```

Animal {

Dog extends Animal {

walk()

bark()

≡

≡

2

3

≡

≡

I) Animal a = new Animal();

a.walk() ✓

a.bark(); ✗

II) Dog d = new Dog();

d.bark() ✓

d.walk() ✓

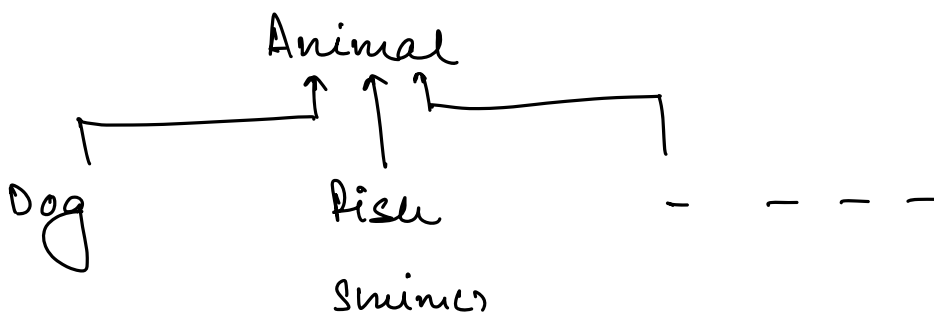
III) Dog d = new Animal(); ✗
d.bark()

→ NOT feasible to
store object of
parent class in the
child class reference.

IV) Animal a = new Dog();

a.walk() ✓

a.bark() ✗



Compile Time Polymorphism.

(Method Overloading).

add(int x, int y) {

return x+y;

2

add(int x, int y, int z) {

return x+y+z;

3

obj.add(2, 5);

obj.add(10, 20, 30);

⇒ fun(String s, int x) { ≡ }

fun(int x, String s) { ≡ }



⇒ Method Signature

method name & data type of input params.

fun(String s, int x) { ≡ }

↳ fun(String, int)

fun(int x, string s) { ≡ }
↳ fun(int, string)

void fun(string s) {
 ≡
 ≡
 ≡
}

→ fun(string)

3

int fun(string s) {
 ≡
 ≡
 ≡
}

→ fun(string)

3

⇒ Compiler doesn't allow methods with same signature in the class.

void fun(string s) {
 ≡
 ≡
 ≡
}

⇒ fun(string)

void fun(int x) {
 ≡
 ≡
 ≡
}

⇒ fun(int)

Run Time Polymorphism.

(Method Overriding)

Class Animal {

void walk() {

}

}

Class Dog extends Animal {

void walk() {

}

int walk() {

return speed;

}

→ walk()

→ walk()

Same
Signature
X

}

→ NOT possible

Class Animal {

Void walk() {

 ①

}

1

Class Dog extends Animal {

Void walk() {

 ②

}

3

1.

Animal a = new Animal()

a.walk() ⇒ ①

2.

Dog d = new Dog()

d.walk(); ⇒ ②

3.

Dog d = new Animal()

d.walk();

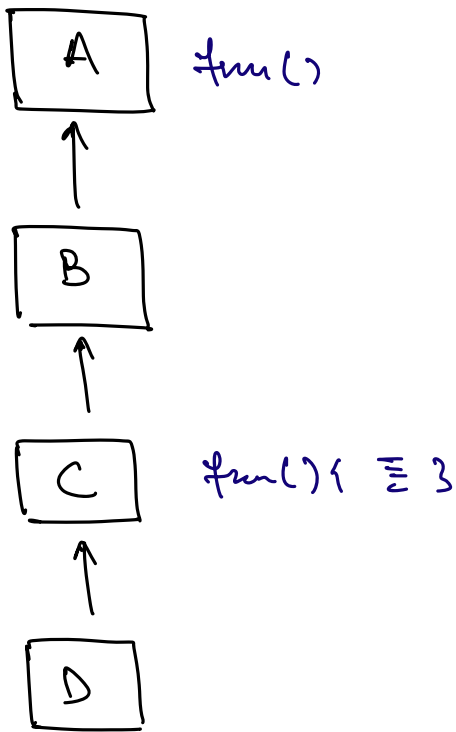
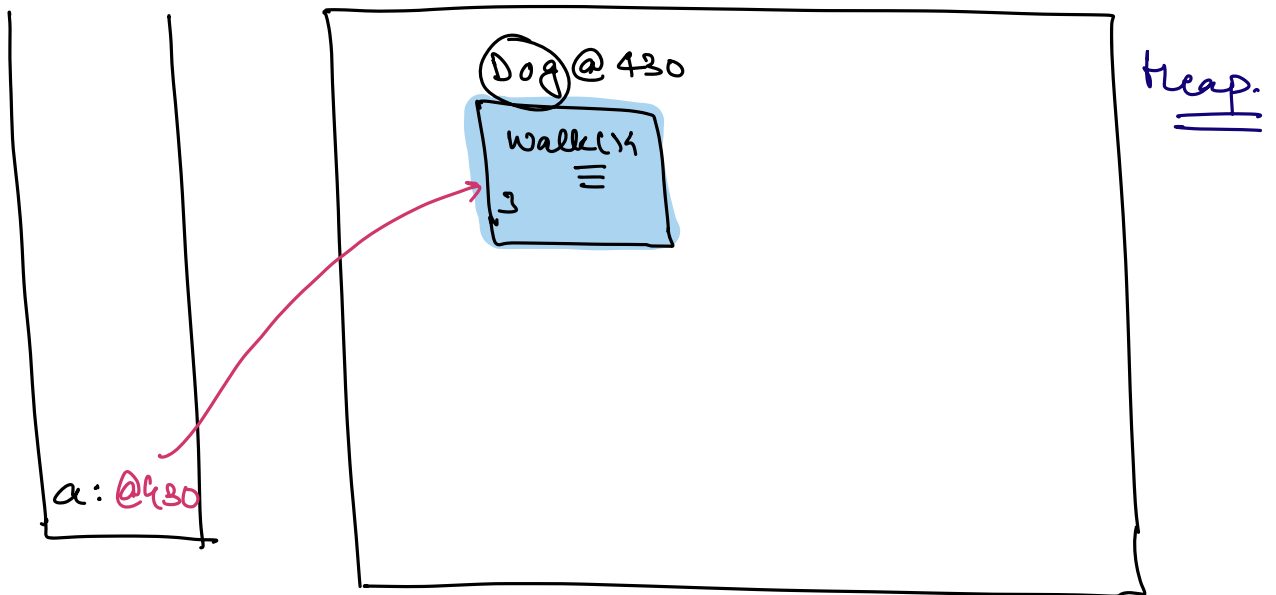


4.

Animal a = new Dog() @430

a.walk()

↳ ②



`D d = new D();`

`d.fun()` $\Rightarrow$ C's fun?

`A a = new C();`

`a.fun()` $\Rightarrow$ C's fun?

`a = new B();`

`a.fun()` $\Rightarrow$ A's fun()

Static keyword

class Student {

name

age

batch

psp

Static

univName;

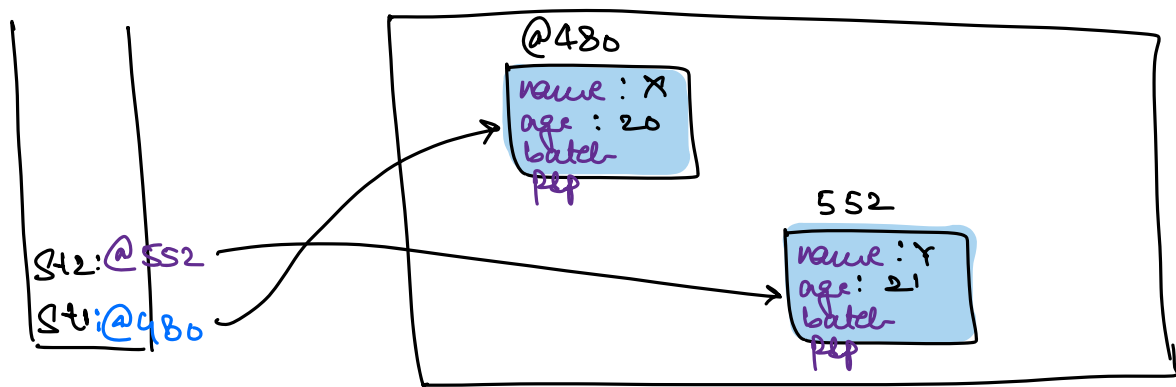
}

Object level attrs.

Class level Attr

Student st1 = new Student();

Student st2 = new Student();



Class
Name

Student.univName = "SST";

⇒ Static attrs are initialized at compile time.

Class Student {

name

age

batch

psp

Static UnivName;

Static

void print() {

==
==
==

}

3

Student.print() ✓

→ Static method can only access static
attrs of the class.

⇒ Non Static method can access both static &
Non Static attrs of the class.

Interfaces.

↳ Blueprint of Behaviours.

interface Runner {

run();

2

↳ Method declaration.

↳ Abstract Method
(Method without body)

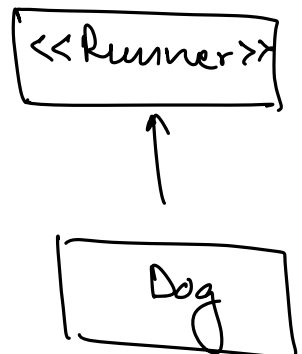
Dog implements Runner {

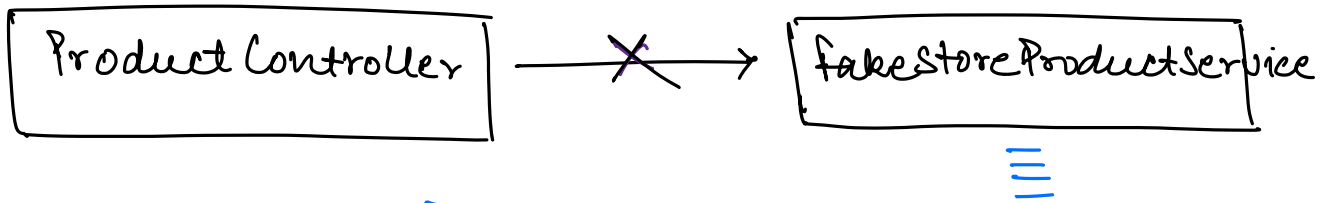
run() {

=====

2

2

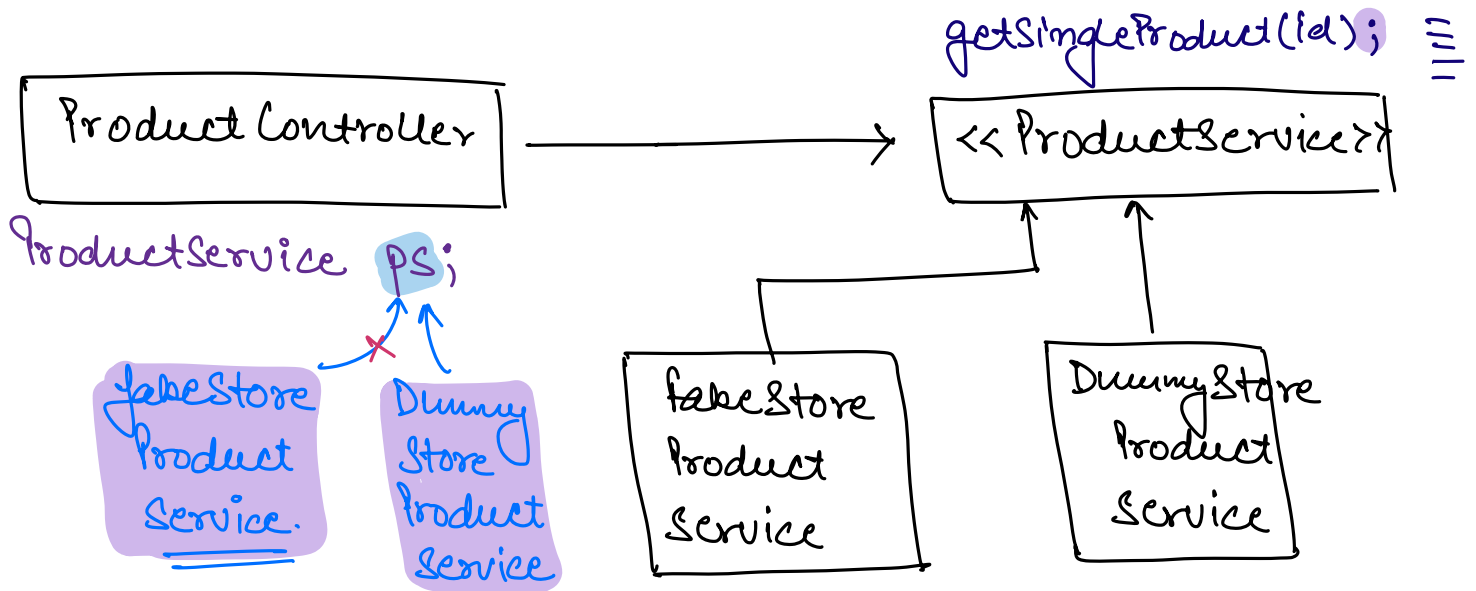




Tight Coupling.

It is difficult to migrate from fakeStoreProductService to some other Service.

⇒ Program to interface, Not to implementation.



ps.getSingleProd(id);

⇒ Loose Coupling

⇒ We can't create of interfaces as they are incomplete.

⇒ In interfaces, attributes are by default public, static & final.

Abstract Class.

↳ Can have abstract methods.